Project Documentation

Reelify: AI-Powered Video to Social Media Reels Converter

Done by:P.Vishnu Priya

TABLE OF CONTENTS

Section	Page No.
Abstract	2
Introduction	3
Scope of Project	4
Requirement Analysis	5–6
Technology Stack	7
Architecture Design	8-9
Development	10
Testing	11–12
Testing Results	13–16
Deployment	17
User Guide	18-19
Future Scope	20
Conclusion	21

Appendices 22-23

References 24

ABSTRACT

This project automates the process of generating short, engaging video reels by allowing users to upload videos or fetch them directly from YouTube. It incorporates a secure local authentication system, enabling users to sign up or log in before accessing the platform. The application processes uploaded or downloaded videos by extracting audio using FFmpeg, transcribing the audio into time-coded text with OpenAI Whisper, and identifying key highlights through a local analysis algorithm. Highlighted segments are cut into reels, resized into the vertical (1080×1920) format, and checked for quality based on configurable duration and resolution criteria. The system is built on a Streamlit-based user interface, providing an intuitive workflow from login to reel generation, preview, and download.

INTRODUCTION

This project provides a streamlined solution for creating short video highlights from longer source videos, aimed at improving content preparation for social media platforms. Users authenticate themselves through a built-in sign-in/sign-up system before accessing any processing features.

Once authenticated, users can either upload a local video file or paste a YouTube link, which the application downloads using yt-dlp. The system uses FFmpeg to extract audio from the video, then transcribes it using the OpenAI Whisper model. A local highlight extraction method is applied to identify the most relevant segments based on transcript length, avoiding reliance on external NLP APIs. These highlights are then cut from the original video and converted to vertical reel format, with automated checks for duration and resolution.

The entire workflow is delivered through an interactive Streamlit interface, ensuring accessibility even for non-technical users. Sidebar controls allow users to set input type, configure quality parameters, and manage authentication, while the main display shows previews, transcripts, and quality reports for generated reels.

SCOPE OF PROJECT

The scope of this project covers the automation of short video reel creation with an integrated authentication mechanism. It caters to content creators, educators, marketers, and individuals seeking to quickly convert long-form videos into ready-to-share short clips.

Key functionalities include:

- User authentication with local storage of credentials.
- Uploading videos in common formats (MP4, MKV, MOV, AVI).
- Downloading YouTube videos directly within the app.
- Extracting and transcribing audio into time-coded text.
- Detecting highlights based on transcript analysis.
- Cutting and resizing reels to a standardized vertical format.
- Performing automated quality checks on duration and resolution.
- Previewing and downloading processed reels from the interface.

Future expansions may include:

- Multi-language transcription support.
- Advanced semantic highlight detection using AI.
- Integration with social media APIs for direct publishing.
- Cloud deployment for multi-user scalability.
- Support for custom background audio tracks.

Requirement Analysis

1.Functional Requirements

- Ability for users to **sign up** with a username, password, and optional display name and email.

- Ability for registered users to **log in** securely before using the application.

- Ability to **upload video files** in formats such as MP4, MKV, MOV, and AVI.

- Ability to **download YouTube videos** within the application using yt-dlp.

- Automatic extraction of audio from the video using FFmpeg.

- Conversion of extracted audio to text with timecodes using OpenAI Whisper's speech-to-text model.

- Identification of key highlights through local transcript analysis based on text segment length.

- Cutting of highlighted segments into reels using FFmpeg's timestamp-based processing.

- Resizing of reels to a standardized vertical format (1080×1920) with padding adjustments.

- Performing **automated quality checks** for each reel, including duration and resolution validation.

- Displaying the generated reels in the application with an option to download them.

2.Non-Functional Requirements

- User credentials must be stored securely using password hashing.
- The system should process videos and generate reels within a reasonable time frame based on system capabilities.
- The interface should be intuitive and easy to navigate for users with minimal technical knowledge.
- The system should be capable of handling variations in video resolution, aspect ratio, and audio clarity.
- Authentication sessions should remain active until the user logs out manually.
- Temporary files should be managed efficiently to avoid unnecessary storage use.

3.User Stories / Use Cases

- As a **new user**, I want to sign up and create an account so I can access the video processing features.
- As a **returning user**, I want to log in securely to my account.
- As a **user**, I want to upload a video file and automatically generate short reels from it.
- As a **user**, I want to paste a YouTube link and generate reels without manually downloading the video.
- As a **user**, I want to preview the reels, check their quality, and download them directly from the interface.
- As a **content creator**, I want the reels to be in vertical format so they are ready for social media sharing.

Technologies and Frameworks

Streamlit

- Selected for its simplicity and rapid development capabilities, allowing quick creation of an interactive user interface without requiring extensive front-end development.
- Used to design the entire application interface, including authentication forms, video upload/download controls, parameter settings, and reel preview displays.
- Limitation: While Streamlit is efficient for rapid prototyping and small-to-medium applications, more complex multi-user applications may require additional frameworks or backend services.

FFmpeg

- Chosen for its versatility in handling various media formats and its ability to perform precise audio extraction, video cutting, and resizing operations.
- Used to extract audio from videos, cut clips by timestamps, and format reels into the 1080×1920 vertical aspect ratio with padding where necessary.
- Limitation: Requires correct local installation and environment setup, which can add complexity for some users.

yt-dlp

- Used to download videos directly from YouTube in supported formats, eliminating the need for manual downloading before processing.
- Ensures compatibility with multiple video qualities and formats.
- Limitation: Dependent on YouTube's changing formats and may require updates over time.

OpenAI Whisper

- Utilized for high-accuracy speech-to-text conversion with timecoded transcripts.
- The lightweight "tiny" model is implemented for faster transcription while maintaining reasonable accuracy.
- Limitation: Accuracy can vary with strong accents, poor audio quality, or background noise.

Python Standard Libraries

- **hashlib**: Used to securely hash user passwords before storing them in the local `users.json` file.
- **json**: Used for reading/writing authentication data and parsing quality-check metadata.
- **os**, **shutil**, **tempfile**: Used for file management, temporary storage, and

  organizing user-uploaded content.

ARCHITECTURE DESIGN

1. Use Case Diagram

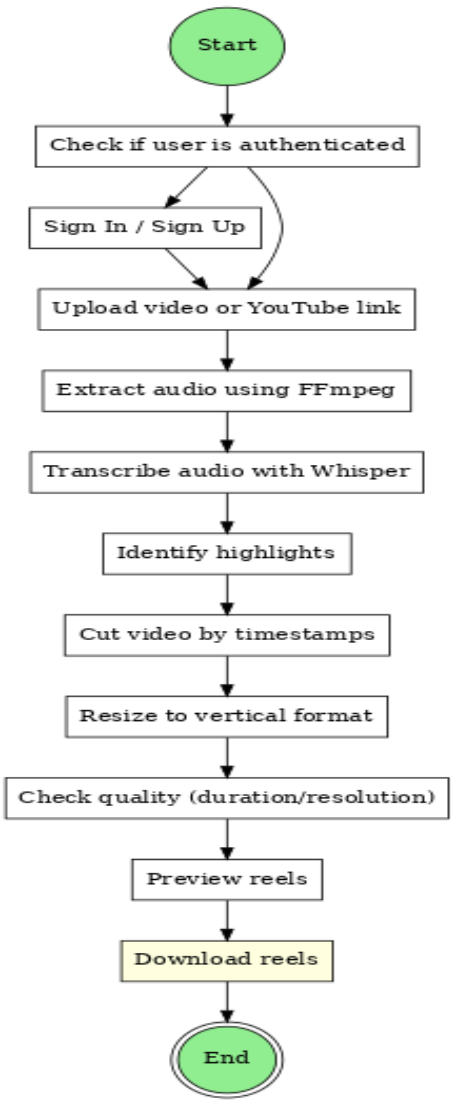
- **Actor:** User
- **Main Actions:**
  - **Register Account** – Sign up with username, password, optional display name, and email.
  - **Log In** – Sign in before using features.
  - **Upload Video / YouTube Link** – Provide video input.
  - **Process Video** – Extract audio, transcribe, find highlights, cut clips, and resize.
  - **Preview & Download Reels** – Watch generated reels, check quality, and save them.

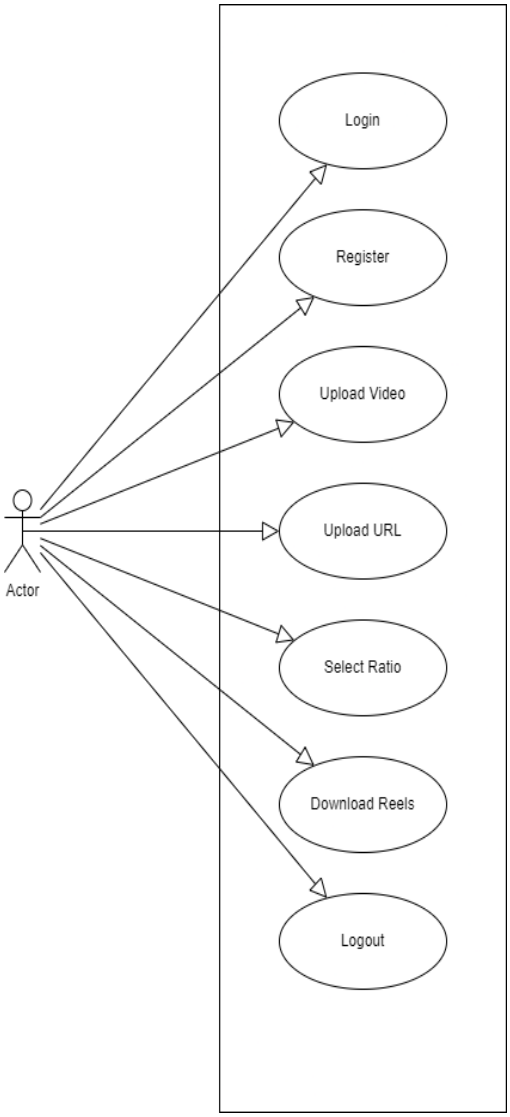
2. Workflow

- **Authentication:**
  - App checks if user is logged in.
  - If not, shows Sign In / Sign Up form in sidebar.
  - Passwords are stored as SHA-256 hashes in `users.json`.
- **Video Input:**
  - Upload a local video (MP4, MKV, MOV, AVI)
  - Or paste a YouTube link (downloaded via `yt-dlp`).
- **Processing Pipeline:**
  - **Audio Extraction** – FFmpeg extracts `.wav` file.
  - **Transcription** – Whisper tiny model creates timecoded text.
  - **Highlight Detection** – Finds top 5 longest transcript segments.
  - **Clip Cutting** – FFmpeg cuts clips using timestamps.
  - **Reel Formatting** – Resizes to vertical (1080×1920) with padding.
- **Quality Check:**
  - Uses ffprobe to check duration & resolution against user settings.
- **Preview & Download:**
  - Shows each reel with preview player and quality check JSON.
  - User can save the reels.

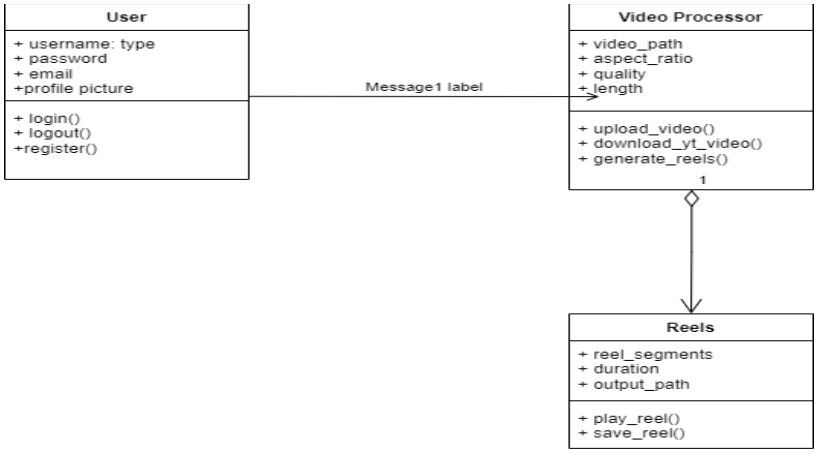
3. Authentication Integration

- **Sign Up:** Saves new account with hashed password, optional display name & email.
- **Sign In:** Verifies entered credentials against stored hash.
- **Session Management:** Keeps login state
- **Logout:** Clears session and authentication info.





Class Diagram



DEVELOPMENT

Technologies & Frameworks

- **Streamlit** – Built the interactive web interface with authentication, video input options, and live previews.
- **FFmpeg** – Managed audio extraction, video cutting, and resizing into vertical (1080×1920) reels.
- **OpenAI Whisper (Tiny Model)** – Transcribed audio into timecoded text for highlight detection.
- **yt-dlp** – Enabled direct downloading of YouTube videos for processing.
- **JSON File Storage (`users.json`)** – Stored user credentials with SHA-256 hashing for authentication without a database.

Coding Standards & Best Practices

- Followed **PEP 8** guidelines for code formatting and readability.
- Designed a **modular structure** separating authentication, video processing, and UI logic.
- Added **inline comments** and descriptive function names for clarity.
- Implemented **robust error handling** and input validation.

Challenges

- Handling **varied video formats** (MP4, MKV, MOV, AVI, YouTube) while maintaining processing consistency.
- Extracting **reliable highlights** without AI model errors.
- Managing **large files** efficiently to avoid performance slowdowns.
- Ensuring **secure authentication** without a full database system.

Solutions

- Developed **dedicated functions** for each step: audio extraction, transcription, highlight detection, cutting, and formatting.
- Chose **top 5 longest transcript segments** for highlights to improve quality without extra API calls.
- Optimized FFmpeg commands for **speed** using ultrafast presets and direct stream copying.
- Added **session-based authentication** with `st.session_state` to maintain login status across actions.

TESTING

1. Unit Testing

Purpose: Verify the correct functioning of each module individually.

Process:

- **Authentication Module:**
  - Test user registration with valid and invalid inputs.
  - Ensure SHA-256 hashing works correctly.
  - Verify login with correct and incorrect credentials.
- **Video Input Module:**
  - Test local uploads for all supported formats.
  - Test YouTube video fetching with valid and invalid URLs.
- **Processing Pipeline:**
  - Validate audio extraction for different formats.
  - Verify transcription accuracy for varied audio clarity.
  - Confirm highlight detection returns correct top 5 segments.
  - Ensure clip cutting produces correct timestamps without corruption.
- **Quality Check Module:**
  - Test ffprobe output for various resolutions and durations.

Outcome: Ensures each module functions independently and returns expected results.

2. End-to-End Testing

Purpose: Validate that the entire application flow works seamlessly from login to reel preview.

Process:

1. Create a new account and log in.
2. Upload an MP4 file and run the full processing pipeline.
3. Repeat with a YouTube link.
4. Check that reels are generated, formatted vertically, and pass quality checks.
5. Verify that session state persists until logout.

Outcome: Confirms smooth integration between modules with no workflow interruptions.

3. Performance Testing

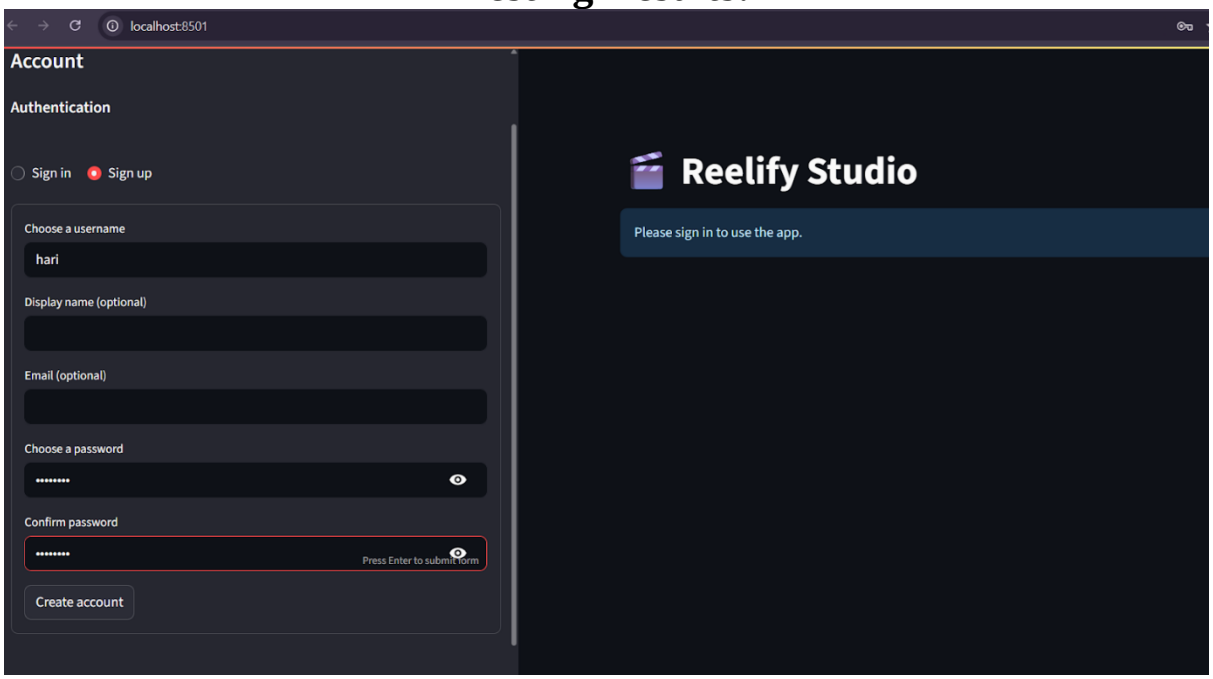
Purpose: Measure application speed and stability under different loads.

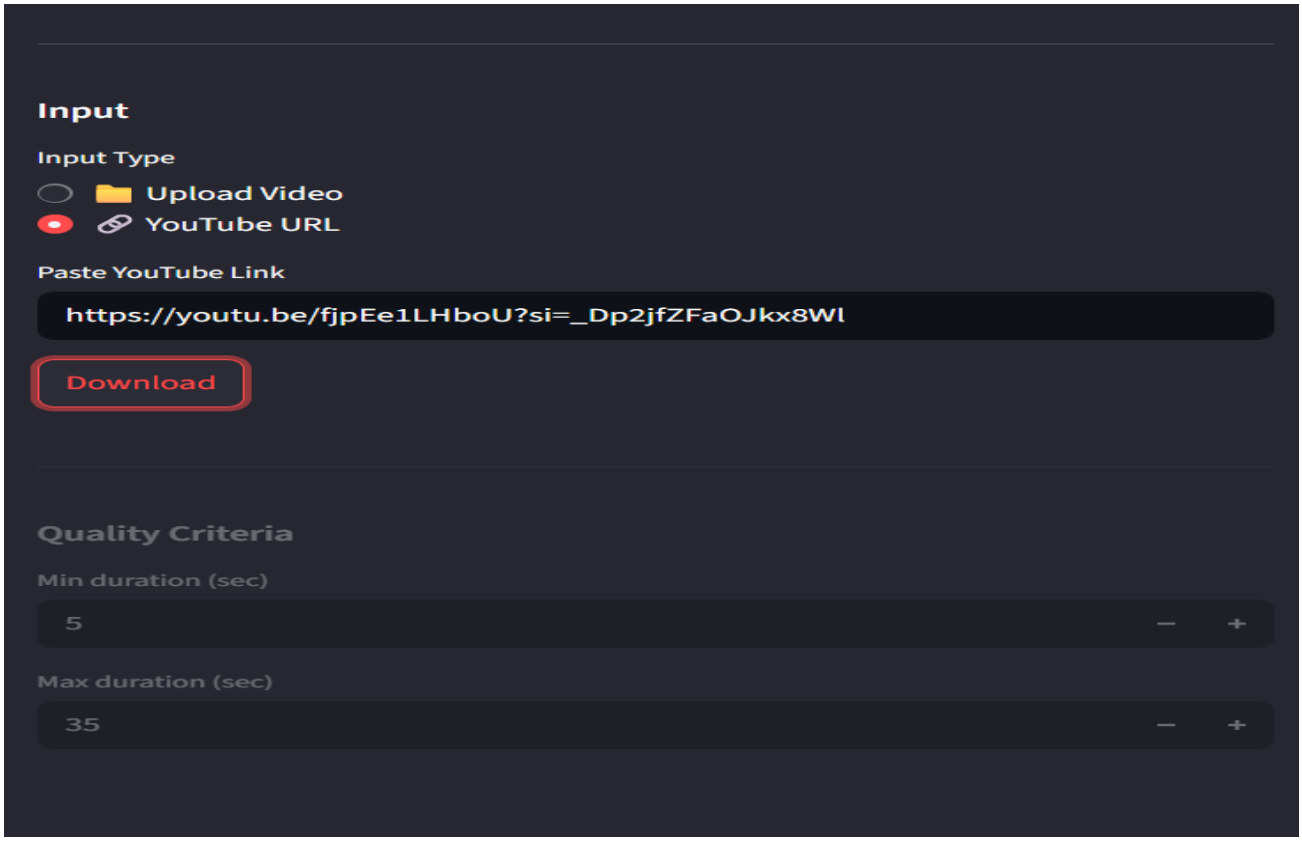
Process:

- Test with videos ranging from 30 seconds to 10 minutes in length.
- Measure processing time for both small (<50MB) and large (>150MB) files.
- Simulate multiple concurrent uploads to evaluate performance impact.

Outcome: Validates that the system remains stable and responsive under expected usage conditions.

Testing Results:




Input
Input Type
Upload Video
YouTube URL
Paste YouTube Link
https://youtu.be/fjpEe1LHboU?si=_Dp2jfZFaOJkx8Wl
Download
Quality Criteria
Min duration (sec)
5
Max duration (sec)
35

0:00 / 1:12

Stop

Deplo

► Run Full Pipeline

0:00 / 1:12

0:00 / 1:12

Transcription Done

Transcript

[00:00:05.800 --> 00:00:08.800] I can use my powers again as before.
[00:00:13.800 --> 00:00:16.800] Women like us before your own path.
[00:00:16.800 --> 00:00:18.800] DeGarrow Grage.
[00:00:18.800 --> 00:00:20.800] Run.
[00:00:22.800 --> 00:00:26.400] You're death is part curse Wednesday.
[00:00:26.400 --> 00:00:28.400] Embrace it.
[00:00:28.800 --> 00:00:31.200] Show me the curse that.
[00:00:31.200 --> 00:00:32.800] What the hell happened?
[00:00:32.800 --> 00:00:36.000] One's the apricot.
[00:00:36.000 --> 00:00:38.800] One's the atoms this year.
[00:00:38.800 --> 00:00:40.600] The atoms with two deeds.
[00:00:40.600 --> 00:00:41.800] Like padded room.
[00:00:41.800 --> 00:00:43.800] Run, run.

```
[00:01:06.800] - [00:01:08.800]: So don't leave the channel before you subscribe.
[00:00:47.400] - [00:00:50.400]: You're talking to worry about me, mother.
[00:00:02.800] - [00:00:05.800]: Thanks to the brain trauma I suffered.
[00:00:22.800] - [00:00:26.400]: You're death is part curse Wednesday.
```

✅ Found 5 valid highlights

🎯 **Reel 1 Quality:**


```
{
  "File" : "reel_1_vertical.mp4"
  "Duration" : "0.0 sec"
  "Resolution" : "1080x1920"
  "Valid Duration" : false
  "Valid Resolution" : true
}
```

🎯 **Reel 2 Quality:**


```
{
  "File" : "reel_2_vertical.m
  "Duration" : "0.5
  "Resolution" : "1080x1920"
  "Valid Duration" false
  "Valid Resolution true
}
```

0:00 / 0:03


```
{
  "File" : "reel_5_vertical.mp4"
  "Duration" : "2.6 sec"
  "Resolution" : "1080x1920"
  "Valid Duration" : false
  "Valid Resolution" : true
}
```


🎉 Successfully created 5 reels!

Deployment

1. Environment Setup

- Install **Python 3.9+**, **FFmpeg**, and **pip** on the target system.

Install dependencies via:
pip install -r requirements.txt

- Ensure ffmpeg and ffprobe are in the system PATH.

2. Application Configuration

- Place users.json in the project directory (auto-generated on first run).
- Set the FFmpeg binary path in the script if using a local Windows installation.
- Optionally, configure environment variables for admin password.

3. Running Locally

Start the app with:
streamlit run app.py

- Access the application at http://localhost:8501.

4. Cloud Deployment (Heroku Example)

- Create a Heroku account and install the CLI.

Initialize Git and push the project:
git init
git add .
git commit -m "Initial commit"
heroku create
git push heroku master

- Add **PostgreSQL** or continue using local JSON for authentication.

Set environment variables for admin credentials:
heroku config:set APP_ADMIN_PASSWORD=yourpassword

5. Testing Post-Deployment

- Verify authentication (Sign Up / Sign In).
- Upload a sample MP4 and run the full processing pipeline.
- Test YouTube link processing and preview generation.
- Check quality settings for min/max duration validation.

USERGUIDE

1. Launching the Application

Run the command: **streamlit run app.py**

- The app will open in your browser at **http://localhost:8501**.

2. Authentication

- **Sign Up**:
  - Go to the sidebar, choose **Sign Up**.
  - Enter a username, password (min 8 characters), and optional display name/email.
  - Click **Create account** to register and log in automatically.
- **Sign In**:
  - Select **Sign In**.
  - Enter registered username and password, then click **Sign in**
- **Logout**:
  - Click **Log out** from the sidebar or top navigation bar.

3. Video Input

After logging in, choose one of the two options from the sidebar:

A. Upload Video

- Click **Upload** and select a file in MP4, MKV, MOV, or AVI format.

B. YouTube URL

- Paste a YouTube video link.
- Click **Download** to fetch the video automatically.

4. Set Quality Criteria

- In the sidebar, set:
  - **Min Duration (sec)** – Minimum allowed length of generated reels.
  - **Max Duration (sec)** – Maximum allowed length of generated reels.

5. Processing a Video

- Click ▶ **Run Full Pipeline**.
- The app will:
  1. **Extract audio** from the video.
  2. **Resize** video to 1080×1920 vertical format.
  3. **Transcribe** audio using Whisper (Tiny model).
  4. **Detect highlights** (top 5 longest transcript segments).
  5. **Cut and format reels** from highlights.
  6. **Check quality** (resolution & duration).

6. Previewing Results

- Each generated reel will be displayed with:
  - **Video preview**.
  - **Quality check details** in JSON format.
- Users can manually save the reels from the preview player (browser's save option).

7. Tips for Best Results

- Use clear audio to improve transcription accuracy.
- Keep YouTube videos under 10 minutes for faster processing.
- Adjust min/max duration to match your desired reel length.

Future Scope of Project

Cloud Deployment

- Host the application on cloud platforms (e.g., Heroku, AWS) for public access without local setup.

Database Integration

- Replace local JSON authentication with a secure PostgreSQL or MongoDB database.

Download Option for Reels

- Add a direct download button for generated reels instead of relying on manual browser saving.

Advanced Highlight Detection

- Use GPT-based semantic analysis for selecting more engaging moments.

Multi-Language Transcription

- Support multiple languages in Whisper for wider accessibility.

User Profile Management

- Allow users to update display name, email, and password from within the app.

Batch Processing

- Enable processing of multiple videos or reels in a single run.

Custom Branding

- Add text overlays, watermarks, and theme customization for social media optimization.

CONCLUSION

This project successfully integrates video processing, audio transcription, highlight detection, and user authentication into a single, user-friendly web application. By combining **Streamlit** for the interface, **FFmpeg** for media handling, **Whisper** for transcription, and a local authentication system, the application delivers an efficient and accessible solution for generating high-quality vertical reels from any video source.

The modular architecture ensures maintainability and scalability, while careful implementation of coding standards and error handling improves reliability. Although the current version uses local JSON storage for user data, it lays a solid foundation for future enhancements such as cloud deployment, database integration, and advanced AI-based highlight detection.

Overall, this project demonstrates the effective use of modern tools and frameworks to automate and simplify a real-world content creation workflow.

APPENDICIES

A. Software Requirements

- **Python Version:** 3.9 or above
- **Dependencies:**
  - streamlit – Web application framework
  - ffmpeg-python – FFmpeg wrapper for Python
  - openai-whisper – Speech-to-text transcription
  - yt-dlp – YouTube video downloader
  - certifi – Certificate bundle for secure HTTP requests
- **External Tools:**
  - **FFmpeg** (with ffprobe) installed and added to system PATH

B. File Structure

```
project_root/
├── app.py              # Main application file
├── requirements.txt      # Dependencies list
├── users.json           # Local user authentication data
├── user_uploads/        # Uploaded videos, audio, and generated reels
└── README.md            # Project overview and instructions
```

C. Sample Test Data

- **Video Formats:** MP4, MKV, MOV, AVI
- **Sample YouTube URLs:**
  - Short clips under 10 minutes for quick testing
  - Public domain videos for demo purposes

D. Authentication Hashing Example

```
import hashlib

password = "mypassword123"

hashed = hashlib.sha256(password.encode("utf-8")).hexdigest()

print(hashed)
```

References

"Whisper: OpenAI's Speech-to-Text Model" (OpenAI Blog) – Official documentation and resources on OpenAI Whisper for speech recognition and transcription.

FFmpeg Documentation – Comprehensive guide for video processing, audio extraction, format conversion, and optimization using FFmpeg.

Streamlit Documentation – Official documentation for building interactive and responsive web applications with Python.

yt-dlp Documentation – Official repository and usage guide for downloading videos from YouTube and other platforms.

GitHub Repositories – Open-source projects and code samples related to video processing, FFmpeg integration, and Streamlit-based interfaces.